

```

import numpy as np

from scipy.fft import fft2, ifft2

from scipy.sparse.linalg import LinearOperator, cg

from PIL import Image

import matplotlib.pyplot as plt

import cv2

import time

import warnings

warnings.filterwarnings("ignore", category=UserWarning, module="matplotlib")

# -----
# Gaussian PSF and FFT utilities
# -----

def gaussian_psf(shape, sigma):
    h, w = shape
    y = np.arange(-(h // 2), h - (h // 2))
    x = np.arange(-(w // 2), w - (w // 2))
    X, Y = np.meshgrid(x, y)
    psf = np.exp(-(X**2 + Y**2) / (2 * sigma**2))
    psf /= psf.sum()
    return psf

def pad_to_image(psf, image_shape):
    ih, iw = image_shape
    ph, pw = psf.shape
    pad = np.zeros((ih, iw), dtype=psf.dtype)
    cy, cx = ph // 2, pw // 2
    pad[:ph, :pw] = psf
    pad = np.roll(np.roll(pad, -cy, axis=0), -cx, axis=1)
    return pad

```

```

def fft_convolve(image, psf_padded):
    return np.real(iff2(fft2(image) * fft2(psf_padded)))

# -----
# FFT Blur Model
# -----

class FTBlurringModel:
    def __init__(self, psf, shape):
        self.psf_padded = pad_to_image(psf, shape)
        self.H = fft2(self.psf_padded)
        self.H_conj = np.conj(self.H)

    def A(self, x):
        return np.real(iff2(fft2(x) * self.H))

    def AT(self, y):
        return np.real(iff2(fft2(y) * self.H_conj))

    def ATA(self, x):
        return np.real(iff2(fft2(x) * (self.H_conj * self.H)))

# -----
# Standard Conjugate Gradient Solver (unchanged)
# -----

def conjugate_gradient_restore(b, model, maxiter=700, tol=1e-6):
    shape = b.shape
    n = shape[0] * shape[1]

    def matvec(x_flat):
        x = x_flat.reshape(shape)

```

```
return model.ATA(x).ravel()
```

```
Aop = LinearOperator((n, n), matvec=matvec, dtype=np.float32)
```

```
rhs = model.AT(b).ravel()
```

```
print("🔧 Running Conjugate Gradient solver...")
```

```
start = time.time()
```

```
x0 = b.ravel()
```

```
sol, info = cg(Aop, rhs, x0=x0, maxiter=maxiter, rtol=tol)
```

```
elapsed = time.time() - start
```

```
if info == 0:
```

```
    print(f"✅ CG converged successfully in {elapsed:.2f}s.")
```

```
else:
```

```
    print(f"⚠️ CG stopped at {maxiter} iterations (info={info}).")
```

```
return sol.reshape(shape)
```

```
# -----
```

```
# 🌟 Spectral Conjugate Gradient (added)
```

```
# -----
```

```
def spectral_conjugate_gradient(b, model, maxiter=700, tol=1e-6):
```

```
    """
```

```
    True Spectral Conjugate Gradient (FFT-based CG)
```

```
    Solves  $(H^*H)x = H^*b$  using CG iterations in the spectral domain.
```

```
    """
```

```
    print("🚀 Running Spectral Conjugate Gradient...")
```

```
H = model.H
```

```
H_conj = model.H_conj
```

```
H2 = H_conj * H #  $|H|^2$ 
```

```

# Right-hand-side:  $AT(b)$ 
rhs = np.real(iff2(H_conj * fft2(b)))

# Initial guess
x = b.copy()

# Residual  $r_0 = rhs - ATA(x_0)$ 
Ax = np.real(iff2(fft2(x) * H2))
r = rhs - Ax
p = r.copy()

r_old = np.sum(r * r)

for k in range(maxiter):

    #  $A_p = ATA(p)$ 
    Ap = np.real(iff2(fft2(p) * H2))

    alpha = r_old / np.sum(p * Ap)

    x = x + alpha * p
    r = r - alpha * Ap

    r_new = np.sum(r * r)
    if np.sqrt(r_new) < tol:
        print(f"✅ Spectral CG converged at iteration {k}")
        break

    beta = r_new / r_old
    p = r + beta * p

```

```

    r_old = r_new

    return np.clip(x, 0, 1)

# -----
# Enhancement Functions (unchanged)
# -----
def enhance_image(img):
    img_uint8 = np.uint8(np.clip(img * 255, 0, 255))

    lab = cv2.cvtColor(img_uint8, cv2.COLOR_RGB2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l2 = clahe.apply(l)
    lab = cv2.merge((l2, a, b))
    enhanced = cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)

    blurred = cv2.GaussianBlur(enhanced, (0, 0), 2)
    sharp = cv2.addWeighted(enhanced, 1.6, blurred, -0.6, 0)

    return np.clip(sharp / 255.0, 0, 1)

# -----
# Restoration Pipeline (ONLY 1 added flag, no logic changed)
# -----
def restore_image_cg_enhanced(image_path, sigma=1.6, maxiter=700):
    img = Image.open(image_path)
    img = img.convert("RGB")
    img = img.resize((256, 256))
    arr = np.asarray(img, dtype=np.float32) / 255.0
    shape = arr.shape[:2]

```

```

psf = gaussian_psf((25, 25), sigma)
model = FFTBlurringModel(psf, shape)

blurred = np.zeros_like(arr)
restored = np.zeros_like(arr)

# 🌟 user can toggle spectral CG here
use_spectral_cg = True

for c in range(3):
    orig = arr[:, :, c]
    blurred[:, :, c] = fft_convolve(orig, pad_to_image(psf, shape))

    if use_spectral_cg:
        restored_channel = spectral_conjugate_gradient(
            blurred[:, :, c], model, maxiter=maxiter
        )
    else:
        restored_channel = conjugate_gradient_restore(
            blurred[:, :, c], model, maxiter=maxiter
        )

    restored[:, :, c] = np.clip(restored_channel, 0, 1 - 1e-8)

enhanced = enhance_image(restored)
return arr, blurred, restored, enhanced

# -----
# Main
# -----

```

```
if __name__ == "__main__":  
    image_path = "crowd.jpg" # <-- Place your image here  
  
    orig, blurred, restored, enhanced = restore_image_cg_enhanced(  
        image_path,  
        sigma=1.6,  
        maxiter=700  
    )  
  
    plt.figure(figsize=(14, 5))  
    plt.subplot(1, 4, 1)  
    plt.imshow(orig)  
    plt.title("Original")  
    plt.axis("off")  
  
    plt.subplot(1, 4, 2)  
    plt.imshow(blurred)  
    plt.title("Blurred Image")  
    plt.axis("off")  
  
    plt.subplot(1, 4, 3)  
    plt.imshow(restored)  
    plt.title("Restored (SCG)")  
    plt.axis("off")  
  
    plt.subplot(1, 4, 4)  
    plt.imshow(enhanced)  
    plt.title("Enhanced Restoration")  
    plt.axis("off")  
  
    plt.tight_layout()
```

```
plt.show()
```

```
out = (enhanced * 255).astype(np.uint8)
```

```
Image.fromarray(out).save("restored_cg_enhanced.png")
```

```
print("💾 Saved enhanced image as 'restored_cg_enhanced.png'")
```